Calculus and Natural Gradient Boosting

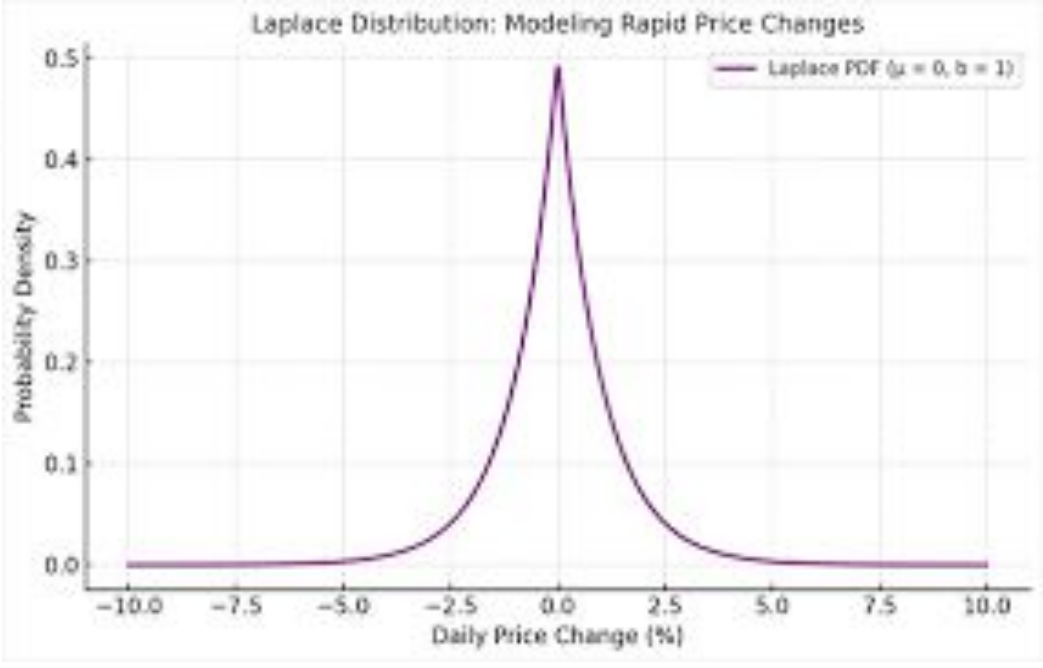
By Eric Zhang

Project Purpose

- Train a machine learning model to predict error in weather forecasting
- Estimate a probability distribution function around the forecast
- Quantify model uncertainty

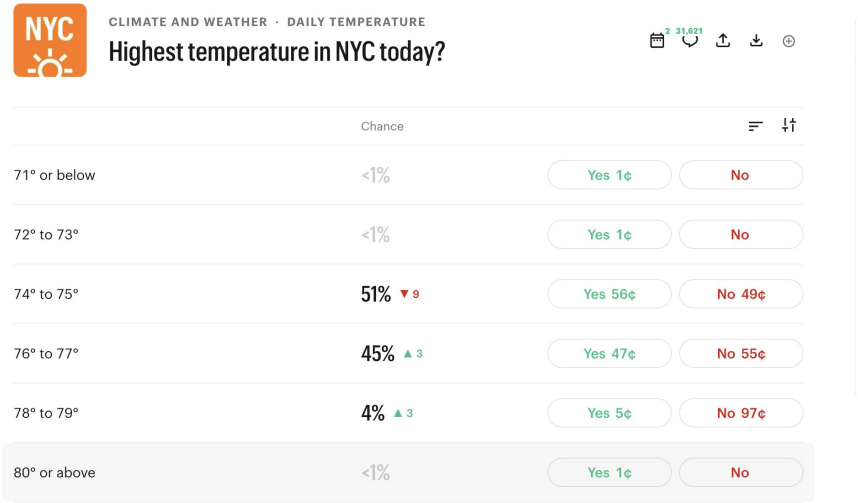
$$F(x) = \int_{-\infty}^{x} f(u)\,\mathrm{d}u = \begin{cases} \frac{1}{2}\exp\left(\frac{x-\mu}{b}\right) & \text{if } x < \mu \\ 1 - \frac{1}{2}\exp\left(-\frac{x-\mu}{b}\right) & \text{if } x \geq \mu \end{cases}$$

$$= \frac{1}{2} + \frac{1}{2}\operatorname{sgn}(x-\mu)\left(1 - \exp\left(-\frac{|x-\mu|}{b}\right)\right),$$



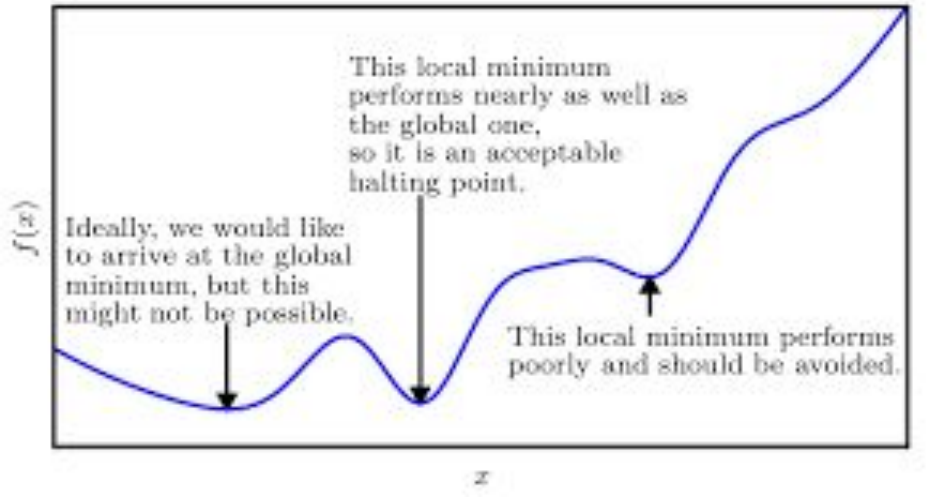
What’s the point

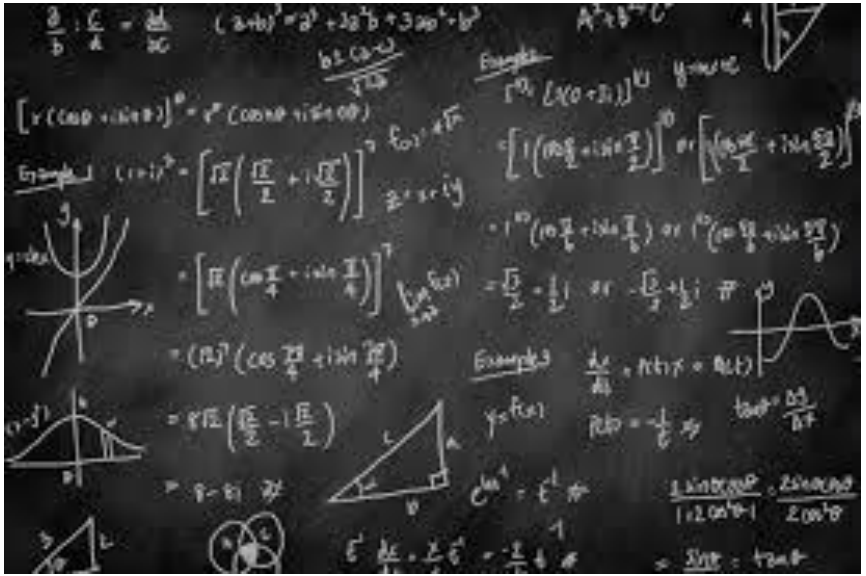
- Forecasts are inherently uncertain
- Distributions allow for greater information and a way to visualize the possibility of tail events.
- Theoretical applications in prediction markets



How is calculus used in this project?

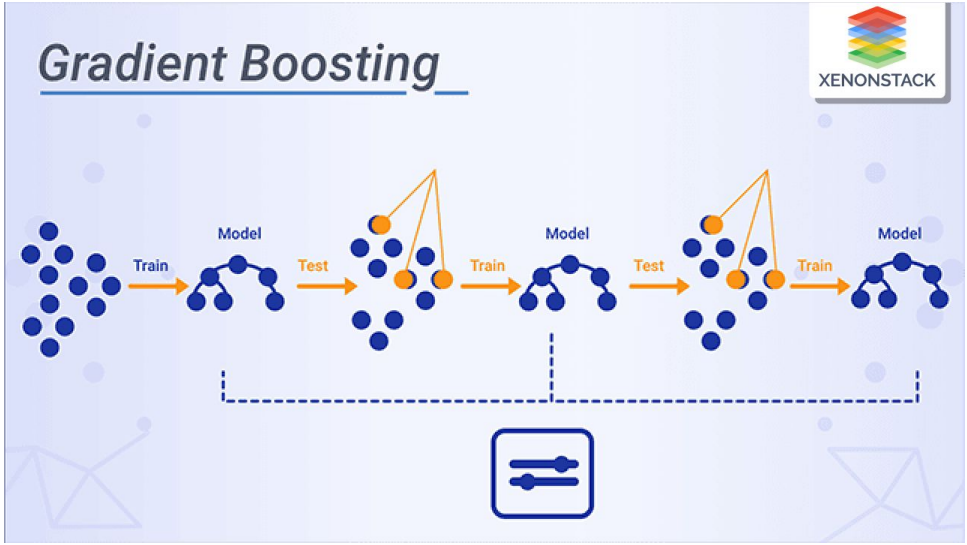
- Machine Learning - relies heavily on calculus to make accurate predictions
- Converting outputs into numbers - In order to get the probability that the temperature falls between a certain interval, you must take the interval of the PDF

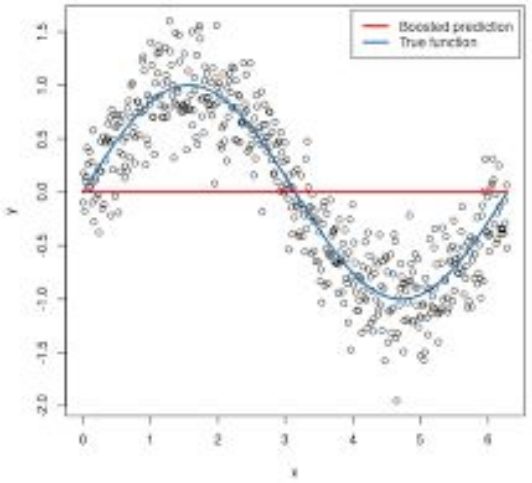




This project uses Natural Gradient Boosting, a more complex variant of Gradient Boosting Machines.

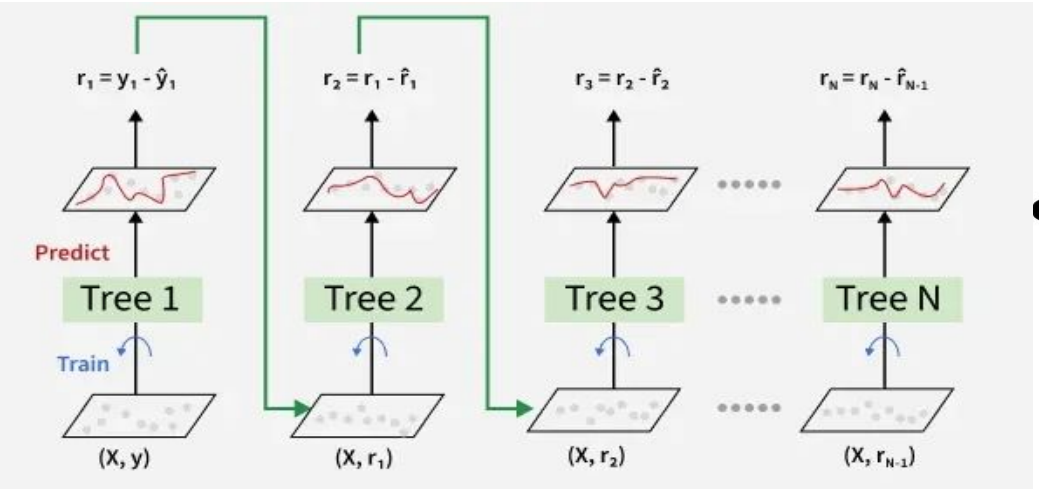
But what are Gradient Boosting Machines?

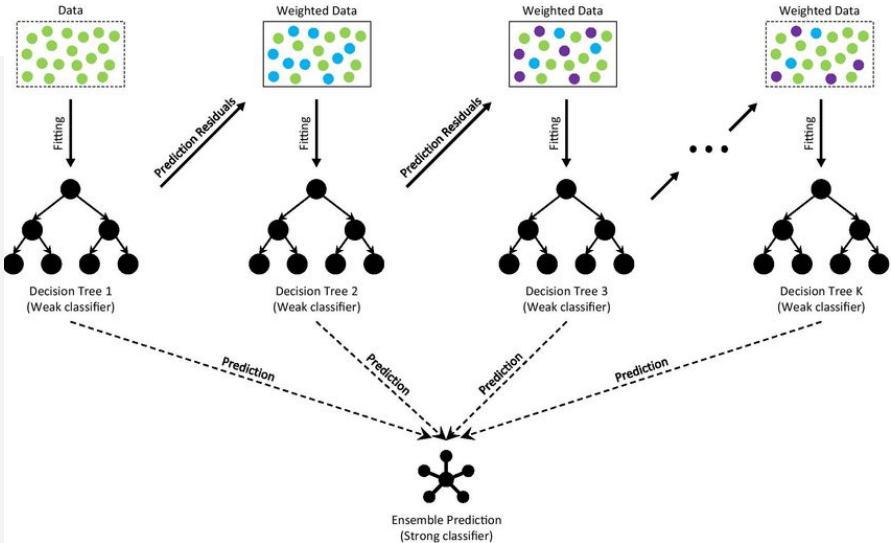




Gradient Boosting Machines

- Machine Learning Model
- Builds trees sequentially
- Each tree builds off of the residuals of the previous tree
- Makes small improvements over hundreds of iterations
- Uses short trees

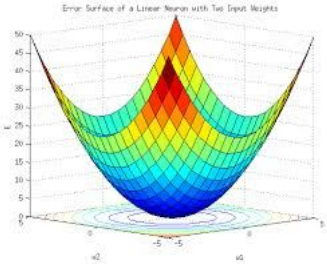




The Math Behind the Model

Step one: The loss function

- Predictions made by the model will always have some degree of error
- Better models have lower loss while worse models have higher loss
- We can minimize this loss by solving an optimization problem, we just need a function to optimize
- We are trying to find the γ that minimizes the sum of the errors across all training samples
- L(y_i,γ) is the loss function we are trying to minimize- what we use depends on the scenario- must be differentiable
- Y_i is just the ith observed value in the sample set of N values



$$F_0(x) = \arg\min_{\gamma} \sum_{i=1}^{N} \mathcal{L}(y_i, \gamma)$$

Ex. Minimizing Log Loss for binary classification

We use this loss function to minimize the log loss $L(p) = -\left[S\log(p) + (N-S)\log(1-p)\right]$

Where $S = \sum_{i=1}^{N} y_i$

S= number of positive examples

N−S = number of negative examples

p = starting probability prediction

We minimize loss by taking the derivative of the loss and setting equal to 0

$$\frac{dL}{dp} = -\frac{S}{p} + \frac{N-S}{1-p}$$

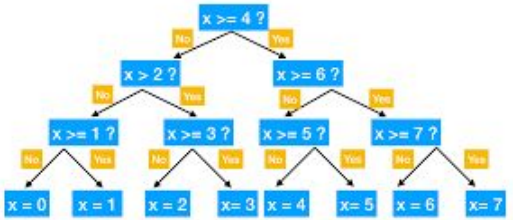
Simplifying, we get

$$p = \frac{S}{N}$$

Step 2: Building the trees

Compute $$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F(x)=F_{m-1}(x)}$$

for i=1,...n



This is the negative gradient of the loss function

M is the current tree we are building

i is the sample number, n is the total number of samples

We get a vector of first derivatives of the loss function, which is our gradient

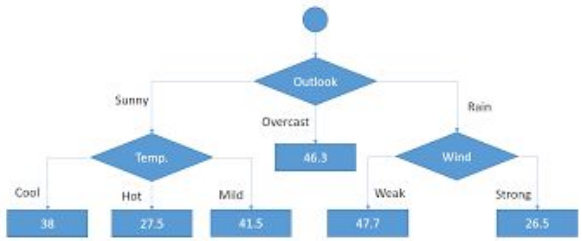
The values of r are called pseudo residuals

(B) Fit a regression tree to the r_{im} and create terminal regions $R_{j,m,}$ for $j = 1...J_m$

Leaves are the terminal regions of R_{jm}

m is the index of the tree we made

j is the index for each leaf in the tree



(C) For $j = 1...J_m$ compute $\gamma_{jm} = \underset{\gamma}{\operatorname{argmin}} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma)$

For each leaf in the new tree, we compute an output value of γ_{jm}

the output value is the γ that minimizes the loss function, taking in the previous prediction into account

only the residuals in each leaf are counted in the minimization

Given the current loss function, the output value is always just the average of residuals in the leaf

(D) Update $F_m(x) = F_{m-1}(x) + \nu \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$

Make new prediction for each sample

Based upon last prediction we made, and the tree we just made

summation used output of the leaf that x belongs to

Add up γ_{jm}'s for all leaves, $R_{jm,}$ that x- a sample- can be found in

ν represents learning rate

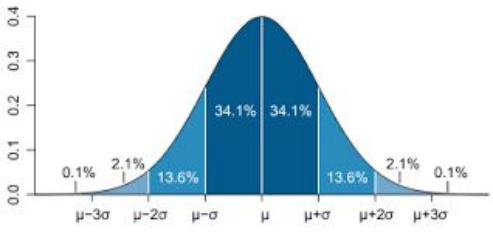
reduces effect of each tree on final prediction

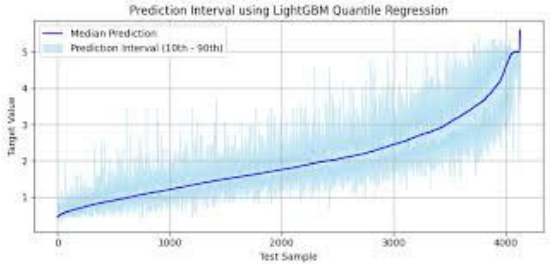
increases accuracy in the long run

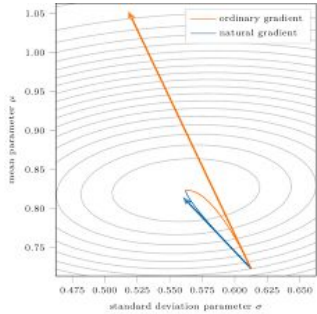
Generates new predictions for each sample

Why do we use natural gradient boosting

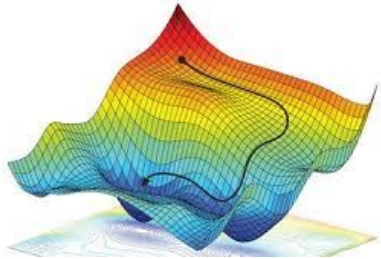
- What are we trying to find? A probability distribution function of forecast error
- Traditional gradient boost methods output either
  - One continuous value - Regression
  - What class the input is in - Classification
- In order to construct a full probability distribution, we would need to train separate models to predict the probability for each interval that we care about
  - But since the models are separate, we cannot guarantee that they agree with each other – what if intervals sum up to a probability greater than 1?
- Natural Gradient Boosting directly outputs a probability distribution, by training separate models for each parameter of the distribution







How do natural gradients change the process



Maps inputs into parameters of a probability distribution function, rather than a singular value

Each iteration updates parameters of the distribution separately

- Separate trees are built for each parameter

Loss function: Minimizing Negative Log Likelihood

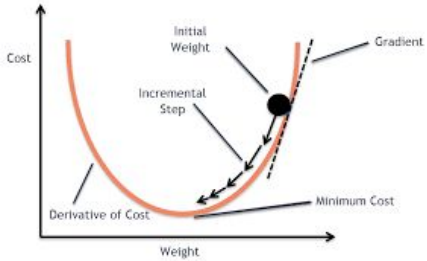
$$\mathscr{L}(\theta) = -\sum_{i=1}^{n} \log P(Y = y_i \mid x_i, \theta)$$

Use natural gradient of normal gradients

- We want the parameter space not the probability space

Why? Standard gradient depends on specific parameters of the PDF

Natural Gradient Boosting: Optimization



Gradient Descent: Take steps in right direction to iteratively find local minimum

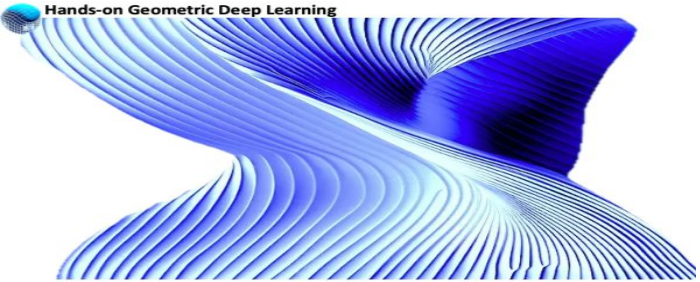
- Use derivatives at each point to calculate which direction reduces loss the most

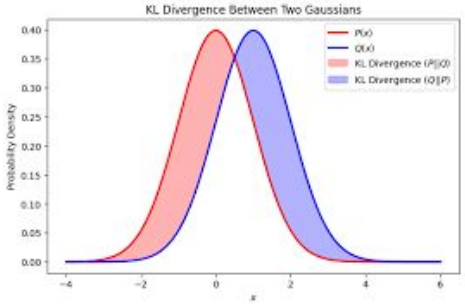
We use natural gradient descent

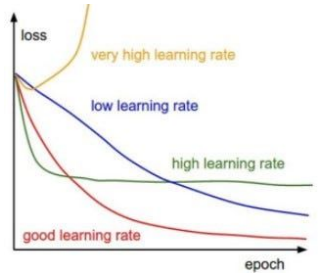
- Instead of finding minimum in a Euclidean space, we optimize on a Riemannian Manifold
- Distance between points on the manifold is the difference between probability distributions with different parameters

We are trying to find an update vector that minimizes the loss function(NLL), that changes the probability distribution up to a limit ε, which is our learning rate

$$D_{KL}(P \parallel Q) = \int_{-\infty}^{\infty} P(x) \log\left(\frac{P(x)}{Q(x)} \right) dx$$







The Math of Optimization

1. Take 1st degree taylor of loss function: $S(\theta + \mathbf{d\theta}, y)$ around $\mathbf{d\theta} = 0$ (dθ is what we are trying to find)
   a. Simplifies to $S(\theta, y) + \mathbf{g}^T\mathbf{d\theta}$ where g is the normal gradient vector
2. The change in the over all distribution is limited by our learning rate, ε, so
   a. $D_{\mathrm{KL}}(P_\theta \parallel P_{\theta+\mathbf{d\theta}}) = \epsilon$ since KL divergence is the difference between two probability distributions
3. We take the 2nd order taylor of the KL divergence $D_{KL}(P_\theta \| P_{\theta+d\theta}) \approx \frac{1}{2} d\theta^T F d\theta$
   a. Where F is the fisher information matrix
      i. Fisher is the hessian of the KL divergence between old and new
4. Given these constraints we can simplify to
   a. $d\theta \propto -F^{-1}g$ Showing that the parameter vector we are trying to find is directly proportional to the negative inverse of the fisher information matrix

$$= -\begin{pmatrix} \frac{\partial^2}{\partial\theta_1^2} & \frac{\partial^2}{\partial\theta_1\partial\theta_2} & \cdots & \frac{\partial^2}{\partial\theta_1\partial\theta_p} \\ \frac{\partial^2}{\partial\theta_2\partial\theta_1} & \frac{\partial^2}{\partial\theta_2^2} & \cdots & \frac{\partial^2}{\partial\theta_2\partial\theta_p} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2}{\partial\theta_p\partial\theta_1} & \frac{\partial^2}{\partial\theta_p\partial\theta_2} & \cdots & \frac{\partial^2}{\partial\theta_p^2} \end{pmatrix} \ell(\theta) \Bigg|_{\theta=\theta^*}$$

Processing outputs

The current model outputs a skew normal distribution

The x axis represents the forecast error

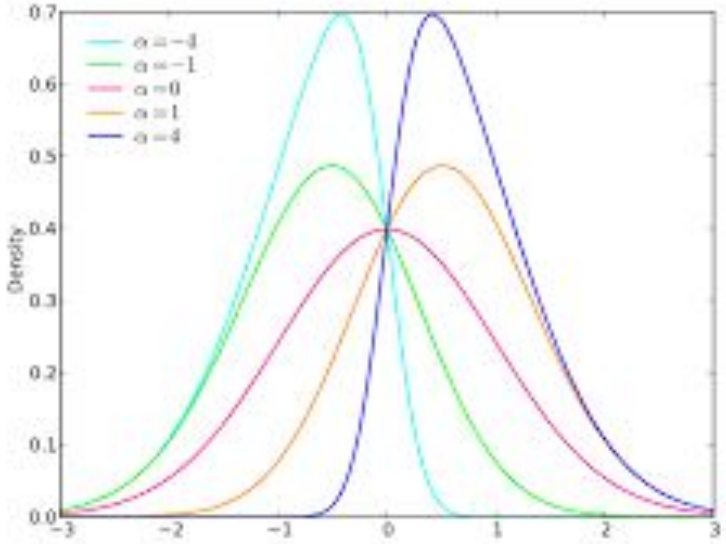
We can find the probability that the forecast

Error is between two values by taking

the integral with bounds set as the

left and right interval

$$\int_a^b f(x)\,dx \qquad f(x) = \frac{1}{2b}\, e^{-\frac{|x-\mu|}{b}}$$



Model Construction Methodology

1. Data collection
2. Data cleaning and organization
3. Define model objective evaluation metrics
4. Construct empirical/normal baseline
5. Selecting a model archetype and feature engineering
6. Train model on data
7. Hyperparameterization, choosing distribution family, and optimization of features
8. Choose best model based upon validation scoring
9. Compare metrics obtained during test period with baseline

Data

Data collected from 3 main sources:

1. National Weather Service: hourly temperature data, daily highs
2. OpenMeteo aggregates weather data from government agencies such as NWS, and the NOAA: live features and climate context
3. National Oceanic and Atmospheric Administration: Used for historical forecasts to base the model off of

Data Cleaning and Processing

Feature list

https://github.com/ericzzhang27-sys/Kalshi-Weather-Trading/blob/main/outputs/day17_feature_lists/full_feature_set.json

Gradient boosting machines have no sense of time, so we give the model context by including features that give time in day and changes in the time series

Other features are included for potential predictive power

Sources

https://arxiv.org/pdf/1910.03225

https://pmc.ncbi.nlm.nih.gov/articles/PMC3873110/

https://www.youtube.com/watch?v=2xudPOBz-vs&list=PLblh5JKOoLUJjeXUvUE0maghNuY2_5fY6&index=2

https://arxiv.org/pdf/2204.00778